ROM

Contents

Cartridge ROM Header

Cartridge Pinout
- Cartridge Bus Timing Diagram

Cartridge ROM Pinout

Checksum (CRC) and hacking difficulties
- Checking Integrated Circuit (CIC) or Lockout chip/boot chip
  - CIC Versions
  - CIC and entry point manipulation
  - Bypassing bad checksum
  - Pinout
- Recalculating CRC

Save Types
- SRAM
- EEPROM
  - Pinout

Backup devices

In order to tell if your ROM is byteswapped: There has always been a push in the Nintendo 64 hacking scene to have all ROMs byteswapped to big-endian. The reason as to why some ROMs are big-endian and some are little-endian is due to backup devices (more pertaining to backup units). With that said, the first word (4-bytes) in every ROM's header should be 80371240 (Sets up the PI so the ROM can be read). If it is not, then it's in little-endian and should be byte swapped before any hacking commences! If your favorite hex editor is worth anything, it should be able to swap bytes. To note, Intel processors, which are of course used in most computers, are little endian.

The N64 accesses data from cartidges and disks through the Parallel interface (registers start at 0xA4600000|0x84600000). For example, you can identify if a compression is used, if it is, of what type and locate the decompression routine by stopping the game every time something is loaded from ROM through the PI registers. Linebreak (breakpoint) write on 0xA460000C, press "step" so the size is set and the interrupt is tripped, then set a read breakpoint on the first few words of data at the RDRAM address being shoved into it. It's fairly straight forward to see how the data is manipulated/treated if the game your hacking has a compression type or not.

Pertaining to file name suffixes for ROM downloads:

```
...............: STANDARD CODES ::...............
:                                               :\
:   [a] Alternate        [p] Pirate             :\
:   [b] Bad Dump         [t] Trained            :\
:   [f] Fixed            [T-] OldTranslation    :\
:                        [T+] NewerTranslation  :\
:   [h] Hack             (-) Unknown Year       :\
:   [o] Overdump         [!] Verified Good Dump :\
:  (M#) Multilanguage (# of Languages)          :\
: (###) Checksum       (??k) ROM Size           :\
:  ZZZ_ Unclassified   (Unl) Unlicensed         :\
:...............................................:\
 \\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\
```

Cartridge ROM Header

The current ROM headers out there are not accurate. Prototypes don't always set an internal name or ID. The IDs are provided by Nintendo when accepting them for release. Moreover, there's a fairly good chance the prototypes you come across that have complete headers had been tempered with (added) by individuals who clean the dumps for certain HLE emulators to be able to run them. The first 0x40 bytes are called the *internal header*, however the N64 never looks beyond 0x18. Between 0x40 to 0x1000 is the bootstrap which is evidently very similar between games. The first 1024 (0x400) bytes of physical memory are reserved for exception vectors and configuration parameters (the interrupt vector table).

The addresses below is only valid for ROM's in Low/High format which is covered in more detail below in the "Backup Devices" category. A "" denotes a note at the bottom.*

Offset	Bytes	Explanation
0x00	x	indicator for endianness (nybble)*
0x01	x	initial PI_BSB_DOM1_LAT_REG (nybble)*
0x01	x	initial PI_BSD_DOM1_PGS_REG (nybble)*
0x02	1	initial PI_BSD_DOM1_PWD_REG*
0x03	1	initial PI_BSB_DOM1_PGS_REG*
0x04	4	ClockRate Override(0 uses default)*
0x08	4	Program Counter*
0x0C	4	Release Address
0x10	4	CRC1 (checksum)
0x14	4	CRC2
0x18	8	Unknown/Not used
0x20	20	Image Name/Internal name*
0x34	4	Unknown/Not used
0x38	4	Media format
0x3C	2	Cartridge ID (alphanumeric)
0x3E	1	Country Code
0x3F	1	Version
0x40	4032	Boot code/strap

Notes:

- *The PI values below will be different depending on the hardware the ROM is encoded on (endianness), e.g. the 64DD IPL and certain development eeproms.*
- *The lower most nybble of the ClockRate is not read.*
- *The program counter (PC) sets the boot location (RAM entry point) when preforming certain kinds of resets(0x8000030C), however some CIC chips will alter the location like how they effect the CRC calculation.*
- *Image Name - max. 20 characters (ascii)*

Space for the game title. Padded with 0x00 or spaces (0x20).
Version: (00 = 1.0, 15 = 2.5, etc.)

- *The boot code is essentially Nintendo's method trying to counter hacking. You can extract these 4032 bytes of data with RN64CRC or other tools. (Covered more below in the CIC section)*

Checksum (CRC)

The checksum is covered more below. Every single ROM has a different checksum value that can be found at 0x10 in it's header. It is calculated from the first megabyte of the game's data and from the CIC-chip checksum value.

Media Format

The media format for the most part is used to matchup save data for specific carts.

```
The first byte indicates the type of ROM
       'N'          cart
       'D'          64DD disk
       'C'          cartridge part of expandable game
       'E'          64DD expansion for cart
       'Z'          Aleck64 cart
The second and third byte form a 2-letter ID for the game
The last byte indicates the region and language for the game
```

Country Code

```
     0x37 '7' "Beta"
     0x41 'A' "Asian (NTSC)"
     0x42 'B' "Brazilian"
     0x43 'C' "Chinese"
     0x44 'D' "German"
     0x45 'E' "North America"
     0x46 'F' "French"
     0x47 'G': Gateway 64 (NTSC)
     0x48 'H' "Dutch"
     0x49 'I' "Italian"
     0x4A 'J' "Japanese"
     0x4B 'K' "Korean"
     0x4C 'L': Gateway 64 (PAL)
     0x4E 'N' "Canadian"
     0x50 'P' "European (basic spec.)"
     0x53 'S' "Spanish"
     0x55 'U' "Australian"
     0x57 'W' "Scandinavian"
     0x58 'X' "European"
```

```
0x59 'Y' "European"
```

Cartridge Pinout

Sadly, I had to sacrifice image clarity in order to fix them within the wiki "barriers." You can click the image to be directed to a larger version.

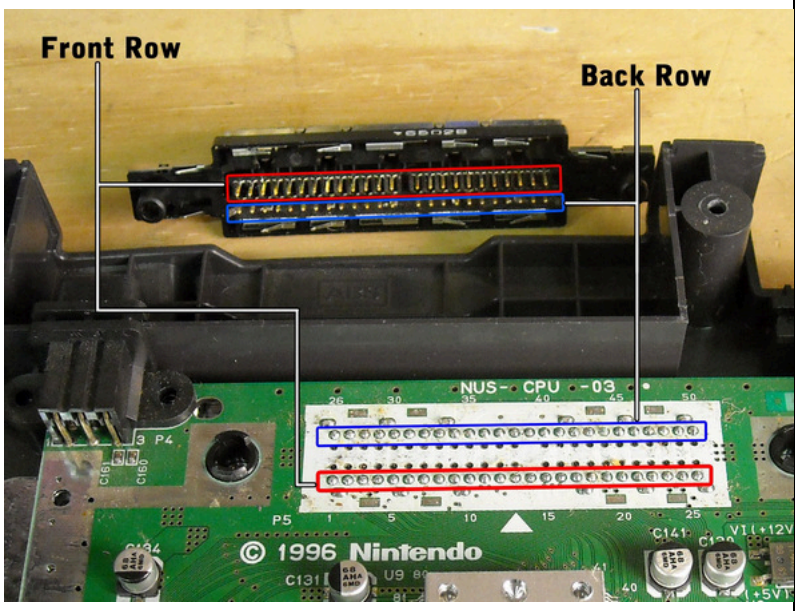
Looking from the top of the console: pin 1 is left front, pin 26 is left rear

- *AD0 - AD15 (I/O): Address Data Bus*

A0-A7: bidirectional signal lines -- serve a dual purpose
A8-A15: unidirectional signal lines -- used as the high - order address bus

- *VCC=3.3V -- Common-collector voltage/Positive supply voltage (VEE being negative, VCC is it's positive equivalent)*
- *nc means Not Connected*

Cartridge Edge (Slot) Pinout



Function	Name	#	-	#	Name	Function
Ground	GND	01	-	26	GND	Ground
Ground	GND	02	-	27	GND	Ground
Address bus	AD15	03	-	28	AD0	Address bus
Address bus	AD14	04	-	29	AD1	Address bus
Address bus	AD13	05	-	30	AD2	Address bus
Ground	GND	06	-	31	GND	Ground
Address bus	AD12	07	-	32	AD3	Address bus
Control	/write	8	-	33	ALE_L	Read Below!
3V3	VCC	9	-	34	VCC	3V3
Control	/read	10	-	35	ALE_H	Read Below!
Address bus	AD11	11	-	36	AD4	Address bus
Address bus	AD10	12	-	37	AD5	Address bus
Address bus	AD9	15	-	40	AD6	Address bus
Address bus	AD8	16	-	41	AD7	Address bus
3V3	VCC	17	-	42	VCC	3V3
CIC to PIF	CIC(15)_TO_PIF	18	-	43	PIF_TO_CIC(14)	PIF to CIC
to PIF	CLK_1.6MHZ	19	-	44	JTAG_CLK_R4300	MIPS related
Reset type	/COLD_RESET	20	-	45	NMI_R4300	NMI Reset type
EPROM	S_DAT	21	-	46	VIDEO_CLK(pin14_vdc)	Not sure
Ground	GND	22	-	47	GND	Ground
Ground	GND	23	-	48	GND	Ground
Left audio	LAUDIO	24	-	49	RAUDIO	Right audio
Ground	GND	25	-	50	GND	Ground

A cool project. (http://bitfixgaming.boards.net/thread/99/bungles-pin-n64-wiring-guide)

Cartridge Bus Timing Diagram

The following images are digital timing diagrams. Timing diagrams are needed to match peripheral devices to microprocessors (the only way to match with microprocessors), and they provide information about different conditions of signals such as high/low, when a machine cycle is being executed.

The Nintendo 64 cartridges uses a multiplex 16-bit bus and just to note, it can be sped up by changing the bus timing (which can be of avail since N64 carts are generally slow). Address multiplexing allows you to use fewer pins on the processor, and thus fewer bus lines. So instead of having some bus lines for the address, and some more for the data, you put the address on the data line, it gets read, then you put the data on the same lines, and it gets read and stored at the previously read address.

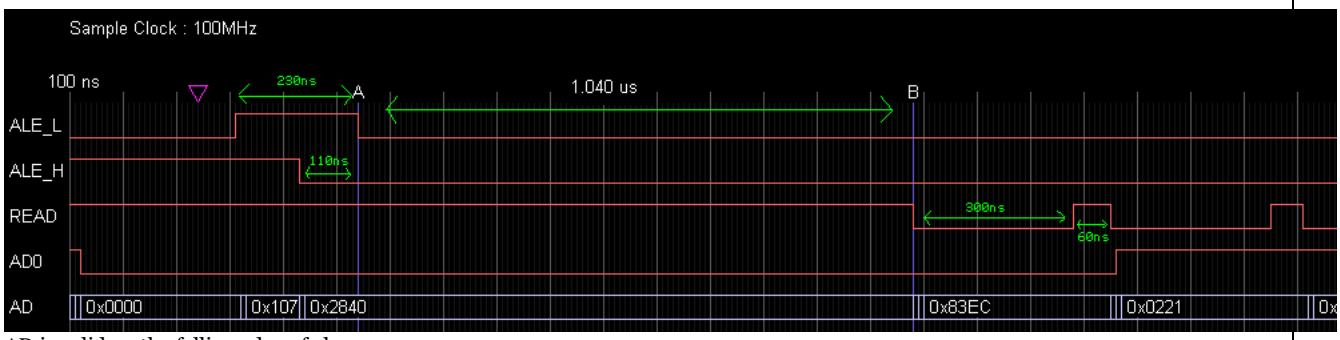
- *ALE stands for Address Latch Enable and is used for external memory interfacing. ALE_L is Address Low and ALE_H Address High. Address Latch Enable is used to demultiplex (reverse process) the address-data signal of port 0 for external memory interfacing. 2 ALE pulses are available for each machine cycle.*

Bus timing:

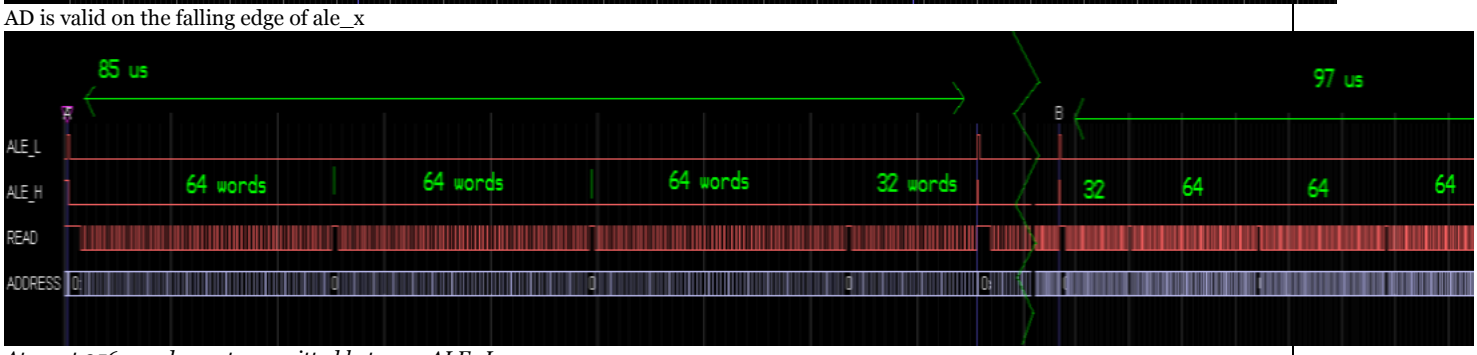
ALE_L	ALE_H	Location
1	1	Cart address (upper 16 bits 0x10xx)
1	0	Cart address (lower 16 bits)

Typical in game bus activity:

- The lines you see running across are called signal lines (in waveform).
- The <--------> you see indicate 1 machine cycle.
- "ns" is the SI unit for nano second.

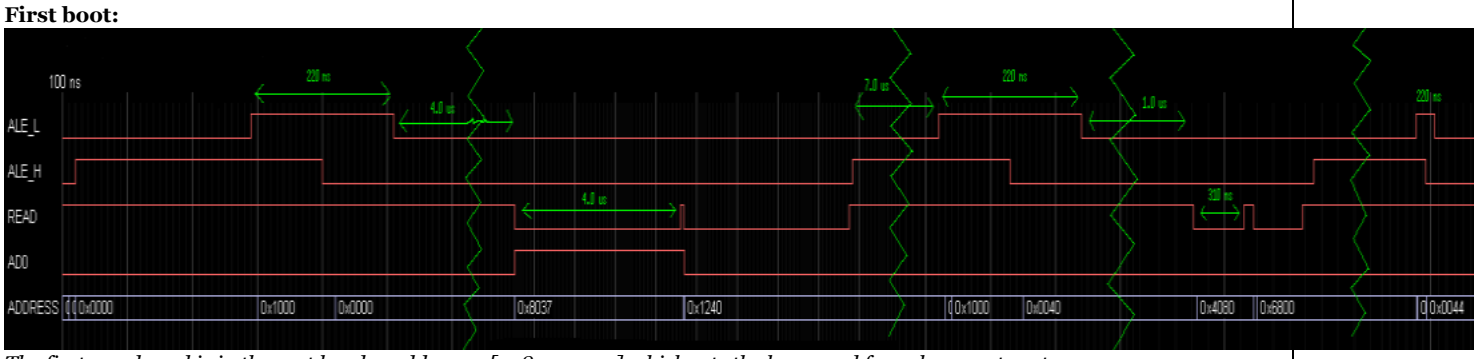


AD is valid on the falling edge of ale_x



At most 256 words are transmitted between ALE_L

First boot:



The first word read is in the cart header address 0 [0x80371240] which sets the bus speed for subsequent cart access.

- byte 0:device latency

- byte 1:release
- byte 2: pulse width
- byte 3:page size

If you set it to 0x8037FF40 then the access timings for loading the boot code at 0x40 in the header would look like the first access at 0, where the read low signal was 4.0us. Can be useful for debugging since all bus activity is slowed down but still functional.

Cartridge ROM Pinout

Name	#	-	#	Name
AD0	1	-	28	AD15
AD1	2	-	27	AD14
AD2	3	-	26	AD13
AD3	4	-	25	AD12
VCC	5	-	24	GND
ALEL	6	-	23	GND
/READ	7	-	22	VCC
GND	8	-	21	nc
ALEH	9	-	20	/READ
GND	10	-	19	VCC
AD4	11	-	18	AD11
AD5	12	-	17	AD10
AD6	13	-	16	AD9
AD7	14	-	15	AD8

Checksum (CRC) and hacking difficulties

This little bugger can be relatively problematic for hackers and modders alike making the big leap from 2D games to more modern 3D games. Granted, checksums had been around for a fair amount of time for copy protection, but in the earlier days of SNES, Sega Genesis and identical systems within that time frame, checksums were a very simple security measure (nothing advanced) and it being wrong/inaccurate had no bearing on gameplay. Every single ROM has a different checksum value that can be found at 0x10 in it's header, while the internal checksum data can be found from 0x1000 to 0x101000. It is calculated from the first megabyte of the game's data and from the CIC-chip checksum value.

On a side not in relation to hacking: Most N64 games follow identical data formats, but with a combination of the checksum, multithreading OS, a myriad amount of compressions, N64 CPU's MMU and TLB (not memory-mapped) hacking Nintendo 64 games can be a lot more difficult/tedious in comparison to older video game systems for those very reasons listed above. The games are not directly memory-mapped from ROM, you cannot use methods of corruption like that with earlier games as you could trigger an access violation by passing the end of a decompression routines buffer which would halt a certain thread, you could break a J instruction and send things like dialog into a permanent loop (However you can locate and NOP the loop), could end up dealing with garbage data, the ROM may not load at all. When it comes to the classical hacking approach of corruption, your best bet is to decompress a ROM first, locate file headers and tinker with them such as changing certain lengths of things. Moreover, your best bet is to learn the low-level language of the Nintendo 64, MIPS.

Checking Integrated Circuit (CIC) or Lockout chip/boot chip

Additional Resource (https://www.retroreversing.com/n64bootcode)

Along with the CRC, you have the lockout chip (CIC or bootchip) which is located in cartridges. The ROM uses the CIC to boot, and to prevent people from copying games and regional importing, and as a side note, has been reported to go "bad" with age. The bootcode calculate and match the result with the CRC's located in the header of a N64 ROM file and if they don't match then the game / demo or whatever refuses to start. For the most part, the CIC is a type of NOR flash memory. The PIF ROM's purpose with this is to validate (first 0x1000 bytes of the ROM) that the ROM has a valid bootcode (CIC) and checksum.

CIC Versions

There are 5 different CIC versions each for NTSC and PAL N64's, and the checksum will only match if the CIC is matched the one the game is expecting. CIC-NUS/boot code types: NTSC-PAL

- Starf-type 6101 (Only Star Fox 64 uses this chip)
- Lylat-type 7102 (Star Fox 64 Pal)
- Mario-type 6102-7101 (e.g. Super Mario 64, CD64 backup unit) //The 6102 chip is considered the standard NTSC security chip.
- Diddy-type 6103-7103 (e.g. Banjo-Kazooie, Paper Mario)
- Zelda-type 6105-7105 (e.g. Zelda: OoT, MM)

- Yoshi-type 6106-7106 (e.g. Yoshi's Story, F-Zero X)

CIC and entry point manipulation

False entry point lookout

As noted above about CIC chips being able to change the RAM entry point as a security measure, and a simple one at that. The 6103 chip adds 0x100000 to the entry location and 6106 adds 0x200000. Below are the individual methods for compensating for the RAM's entry point if the CIC does in fact relocate the entry address (The following is *STRICTLY* for CIC-NUS and may be different for other regions):

- 6101 Doesn't relocate
- 6102 Doesn't relocate
- 6103 Subtract 0x100000 (0x80100400 to 0x80000400)
- 6105 Doesn't relocate
- 6106 Subtract 0x200000 (0x80200400 to 0x80000400)

Bypassing bad checksum

Getting around checksum issues

The checksum trap can be bypassed by NOPing the following two bootcode instructions. Their location in ROM varies depending on CIC type.

```
BNE A3, T0, 0x800001C8
BNE S0, T0, 0x800001C8
```

CIC	Address 1	Address 2
6101	0x670	0x67C
6102-7101	0x66C	0x678
6103-7103	0x63C	0x648
6105-7105	0x77C	0x788

The emulator will complain about it ("unknown CIC"), but it works.This is useful when modying stuff in the checksum area.

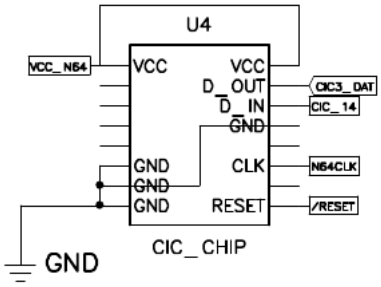
- 1964 - Doesn't complain at all.
- Mupen 0.5.1 - Same
- Mupen64Plus 2.0 - Same
- Nemu64 0.8 - One error message ("Unknown bootcode. Chances that this game won't work are high").
- Project 64 1.6 - Two error messages ("Unknown Cic Chip" and "Unhandled CicChip(-1) in first DMA".

For further elaboration on entry points

Pinout

Just a note: It is to my understanding that you can order the EverDrive 64 without a CIC chip at a cheaper price. I don't recommend attempting to solder CIC chips if you're truly an amateur solderer, if that's applicable to you, the reader, in the mean time, checkout this (https://www.youtube.com/watch?v=5uiroWBkdFY) awesome video (Read: SMD soldering not related; just an amazing video).

U4 is some kind of manufacture identification prefix.



Name	#	-	#	Name
VCC	1	-	16	VCC
nc	2	-	15	DATA_OUT
nc	3	-	14	DATA_IN
nc	4	-	13	GND
nc	5	-	12	nc
GND	6	-	11	CLK(~1.5MHZ)
GND	7	-	10	nc
GND	8	-	9	/COLD_RESET

- CLK = Clock in (an oscillating signal) (provides the basic timing for the processor and bus controller)

A fun project would be ordering you a toggle switch to shift between two different chips (I think someone did this on benheck or ASSEMbler games).

Recalculating CRC

To recalculate cartridge header checksums use this tool (http://n64dev.org/n64crc.html). You can also recalculate it in Nemu64: Open the Memory editor window before the desired game is ran and set a debugger 'read' linebreak at 0x10000010, load the game and write the generated values from the registers to the ROM when the program halts. I'd recommend the former method.

Save Types

Save types

- 4 Kilobits EEPROM
- 16 Kilobits EEPROM
- 32 Kilobytes SRAM (Only twelve games use this type of save)
- 1024 Kilobytes FLASH
- 32KBytes Controller Pak SRAM (Secondary save)

SRAM

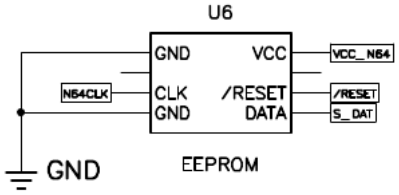
Games that use this save store the save data on a "CR2032" battery which is easy to replace!

EEPROM

EEPROM stands for: Electrically Erasable Programmable Read-Only Memory. It's a non-volatile memory type and is used primarily to store small amounts of data even in applications outside the Nintendo 64, an it takes approximately 15 milliseconds for the data to actually be written to the memory cell and the chip can handle approximately 100,000 write cycles. Most Nintendo 64 games utilize eeprom and can be found in both the Controller Pak and in cartridges. Moreover, there are two types of EEPROM which only differ in capacity: 4k EEPROM (4 kilobits (64 words * 64 bits = 512 bytes) e.g. Banjo-Kazooie) and 16k EEPROM (16 kilobits (256 words * 64 bits = 2048 bytes) e.g. Banjo-Tooie) but only ten games utilize the later.

Pinout

U6 is some kind of manufacture identification prefix.



Name	#	-	#	Name
GND	1	-	8	VCC
nc	2	-	7	nc
CLK	3	-	6	/COLD_RESET
GND	4	-	5	DATA

- CLK = Clock in (an oscillating signal) (provides the basic timing for the processor and bus controller)

Backup devices

One of the greatest things about the society we live in today is the balance of forces. For every action, there is an equal and opposite counteraction. The reaction to the N64 was lukewarm for many people who believed that there were more advantages to the CD-ROM format, and were irate at the high prices that Nintendo charged for its games. Thus, the console got a number of Taiwanese and HK manufacturers to build units that would interface with the N64 console and emulate a game cartridge that wasn't really there.

The four N64 backup units in wide use are the: Doctor V64 and V64jr by Bung, the CD64 by UFO Company, and the Z64 by Interesting Devices. All of these units are very handy for a N64 user. Even though they were primarily designed to play pirated games, they have many other legitimate uses. Homebrew development is one. Not every aspiring game developer can afford the official Nintendo dev kits. With a backup unit, anyone can create their own programs and run them on the N64. Also, cheat codes and save management are two other widely used features on backup units. Another legitimate use (but still one Nintendo would not like you to do) is to play import game cartridges, which are normally locked-out from the N64 booting them.

Backup unit users are really not all the scumbag pirates that Nintendo would have you believe. As CrowTRobo of Obsidian would agree, the people that have spent money on a backup unit also must own a real N64, and thus are much more likely to buy original Nintendo software than the idiots who swarm around the N64 emulation "scene" and download entire GoodN64 collections. Typically, backup unit users are a more tech-savvy crowd than the typical consumer, and thus they are the people that can benefit the most from having access to the neat features that backup units offer. In addition, Nintendo benefits from having a homebrew development scene and having more people familiar with coding for its console. Nintendo would not agree with this, but I firmly believe that a backup unit is just one more reason to own a N64, and thus helps Nintendo more than it hurts in the long run, as a console's install base is much more important than individual game sales ever will be.

Endianness

- Mr Backup Z64 (.BIN or .Z64 files) uses a Low/High byte format

(Big endian), wich is the "correct" format to read from.

- Doctor V64 (.V64, .N64 or .U64 files) uses a High/Low byte format

(Middle endian), so each word are byte flipped, like this:
"ETTSNI G"
When it should look like this:
"TESTING "
To solve this, rotate each word by 8 bits.

Differences between Z64, N64, V64

Problems we encounter with backup units
The above is a direct copy from the n64 subdomain of icequake. For information pertaining to technical information on the backup units as well as problems with backup units, click me! (http://n64.icequake.net/#backup)

A special thanks to the now defunct crazynation.org for the pinouts and the digital timing diagrams!!!

Retrieved from "https://en64.shoutwiki.com/w/index.php?title=ROM&oldid=539"

This page was last edited on 17 November 2019, at 06:37.

Content is available under Creative Commons Attribution-Share Alike 3.0 unless otherwise noted.